



Exercício: Utilizando a palavra-chave `super`

Objetivo

Entender como a palavra-chave `super` é usada para chamar métodos da superclasse em Java.

Instruções

1. Crie uma classe chamada `Veiculo` com um método chamado `acelerar()`. O método `acelerar()` deve imprimir "Veículo acelerando!".
2. Crie uma subclasse chamada `Carro` que estenda a classe `Veiculo`.
3. Na classe `Carro`, sobrescreva o método `acelerar()` para imprimir "Carro acelerando!".
4. Utilize a palavra-chave `super` para chamar o método `acelerar()` da superclasse `Veiculo` dentro do método sobrescrito da classe `Carro`.

Passos para fazer o exercício

1. Crie as classes `Veiculo` e `Carro`.
2. Implemente o método `acelerar()` na classe `Veiculo`.
3. Sobrescreva o método `acelerar()` na classe `Carro` e utilize `super.acelerar()` para chamar o método da superclasse.
4. Crie um objeto da classe `Carro` e chame o método `acelerar()`.

Código do exercício

```
class Veiculo {
    void acelerar() {
        System.out.println("Veículo acelerando!");
    }
}

class Carro extends Veiculo {
    @Override
    void acelerar() {
        System.out.println("Carro acelerando!");
        super.acelerar(); // Chamando o método da superclasse
    }
}

public class Main {
    public static void main(String[] args) {
        Carro meuCarro = new Carro();
        meuCarro.acelerar();
    }
}
```

Explicação do código

- A classe `Veiculo` possui um método `acelerar()` que imprime uma mensagem.
- A classe `Carro` estende `Veiculo` e sobrescreve o método `acelerar()`, adicionando uma mensagem específica para carros.
- Dentro do método sobrescrito da classe `Carro`, utilizamos `super.acelerar()` para chamar o método da superclasse `Veiculo`.



Exercício: Aplicando Herança

Objetivos:

O objetivo deste exercício é praticar conceitos de herança em Java. Vamos criar uma classe `Assistente` que herda da classe `Funcionario` e implementar um método para calcular o salário anual com um bônus fixo.

Instruções:

1. Crie as classes `Funcionario` e `Assistente`.
2. Implemente os métodos necessários para calcular o salário anual.
3. **Como os atributos serão `private`, crie os getters e setters necessários para acessá-los.**
4. No método `main` da classe `TesteHeranca`, crie um objeto `Assistente`, defina seu nome, salário e aumento.
5. Exiba o nome e o salário anual do assistente.

Passos para fazer o exercício:

1. Implemente as classes `Funcionario` e `Assistente`.
 2. Na classe `Funcionario`, crie:
 - os atributos `nome` e `salario` como `private`
 - os métodos:
 - `addAumento(double valor)`
 - `ganhoAnual()`
 - **getters e setters para `nome` e `salario`**
 3. Na classe `Assistente`, herde da classe `Funcionario` e sobrescreva o método `ganhoAnual()` para adicionar um bônus fixo de R\$ 1000.
 4. No método `main` da classe `TesteHeranca`, crie um objeto `Assistente`, defina:
 - nome como `"João"`
 - salário como `3000`
 - aumento como `500`
 5. Exiba o nome e o salário anual do assistente usando `System.out.println`.
-

Código do exercício

```
class Funcionario {

    private String nome;
    private double salario;

    public void addAumento(double valor) {
        salario += valor;
    }

    public double ganhoAnual() {
        return salario * 12;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }

    public void setSalario(double salario) {
        this.salario = salario;
    }
}

class Assistente extends Funcionario {

    private int numeroMatricula;

    public double ganhoAnual() {
        // Considerando um bônus fixo de R$ 1000 para assistentes
        return super.ganhoAnual() + 1000;
    }
}

public class TesteHeranca {

    public static void main(String[] args) {

        Assistente assistente = new Assistente();

        assistente.addAumento(500); // Aumento de R$ 500
        assistente.setNome("João");
        assistente.setSalario(3000);

        System.out.println("Nome: " + assistente.getNome());
        System.out.println("Salário anual: R$" + assistente.ganhoAnual());
    }
}
```

Explicação do código:

- A classe `Funcionario` possui os atributos `nome` e `salario` definidos como `private`.
- Para acessar esses atributos, foram criados **getters e setters**, seguindo o encapsulamento.
- A classe `Assistente` herda da classe `Funcionario` e sobrescreve o método `ganhoAnual()`, utilizando `super` para reaproveitar o cálculo da classe pai.
- No método `main`, os dados do assistente são definidos usando os métodos públicos da classe.
- O resultado é exibido no console com o nome e o salário anual.



Exercício Complementar: Herança e uso da palavra-chave `super`

Objetivo

Demonstrar como a palavra-chave `super` é usada para acessar membros da classe pai (superclasse) em uma subclasse.

Instruções

1. Crie duas classes: `Pessoa` (superclasse) e `Estudante` (subclasse).
2. A classe `Pessoa` deve ter os seguintes atributos: `nome` e `idade`.
3. A classe `Estudante` deve estender `Pessoa` e ter um atributo adicional chamado `matricula`.
4. Implemente construtores para ambas as classes.
5. Na classe `Estudante`, use a palavra-chave `super` para chamar o construtor da classe pai (`Pessoa`) e inicializar os atributos herdados.
6. No método `main`, crie um objeto `Estudante` com informações de um estudante e exiba os detalhes (nome, idade e matrícula).

Passos para fazer o exercício

1. Defina a classe `Pessoa` com os atributos `nome` e `idade`.
2. Implemente um construtor na classe `Pessoa` que aceite valores para `nome` e `idade`.
3. Crie a classe `Estudante` que estende `Pessoa` e adicione o atributo `matricula`.
4. No construtor da classe `Estudante`, utilize `super(nome, idade)` para chamar o construtor da classe pai.
5. No método `main`, crie um objeto `Estudante` com informações fictícias e exiba os detalhes.

Código do exercício

```
class Pessoa {
    private String nome;
    private int idade;

    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    public String getNome() {
        return nome;
    }

    public int getIdade() {
        return idade;
    }
}

class Estudante extends Pessoa {
    private int matricula;

    public Estudante(String nome, int idade, int matricula) {
        super(nome, idade); // Chama o construtor da classe pai
        this.matricula = matricula;
    }

    public int getMatricula() {
```

```
        return matricula;
    }
}

public class Principal {
    public static void main(String[] args) {
        Estudante estudante = new Estudante("João", 20, 12345);
        System.out.println("Nome: " + estudante.getNome());
        System.out.println("Idade: " + estudante.getIdade());
        System.out.println("Matricula: " + estudante.getMatricula());
    }
}
```

Explicação do código

- A classe `Estudante` herda os atributos `nome` e `idade` da classe `Pessoa`.
- O construtor da classe `Estudante` chama o construtor da classe pai usando `super(nome, idade)`.
- O método `main` cria um objeto `Estudante` com informações fictícias e exibe os detalhes. 😊